

UNIVERSIDAD AMERICANA



Algoritmo y Estructura de Datos

Analisis de algoritmo de ordenacion y busqueda

Estudiantes:

Allan Raúl Valentín Acuña Otero **CIF:** 23020611

Fernando Gabriel Gómez Gazo **Cif:** 23012349

Guillermo José Vega Vega **Cif:** 24010412

Gabriela Michelle Guerrero Paiz **Cif:** 24010347

Steven Leonel Sequeira Reyes **Cif:** 24010446

Docente:

Jose Alejandro Durán

Fecha:

23 de Junio del 2025

Índice

Algoritmos de ordenación.....	3
Ordenamiento de burbuja (Bubble Sort).....	3
Ventajas.....	3
Desventajas.....	4
Casos de uso.....	4
Ordenamiento por mezcla (Merge Sort).....	4
Ventajas.....	5
Desventajas.....	5
Casos de uso.....	5
Ordenamiento rápido (Quicksort).....	5
Ventajas.....	6
Desventajas.....	6
Casos de uso.....	7
Algoritmos de búsqueda.....	8
Búsqueda lineal (secuencial).....	8
Ventajas.....	8
Desventajas.....	8
Casos de uso.....	9
Búsqueda binaria.....	9
Ventajas.....	10
Desventajas.....	10
Casos de Uso.....	10
Búsqueda en grafos.....	11
Búsqueda en anchura (BFS).....	12
Ventajas.....	12
Desventajas.....	12
Casos de Uso.....	13
Búsqueda en profundidad (DFS).....	13
Ventajas.....	14
Desventajas.....	14
Casos de Uso.....	14
Análisis a posterior.....	15
Algoritmos de ordenamiento.....	15
Algoritmos de búsqueda.....	16

Algoritmos de ordenación

Los algoritmos de ordenación reorganizan los datos de una colección según un criterio (por ejemplo, numérico o alfabético). Entre los más conocidos se encuentran bubble sort, mergesort y quicksort. A continuación se describen sus definiciones, funcionamiento, ejemplos sencillos y casos de uso típicos.

Ordenamiento de burbuja (Bubble Sort)

Es un algoritmo muy simple que recorre repetidamente la lista comparando pares de elementos adyacentes e intercambiándolos si están en orden incorrecto. En cada pasada, el elemento más grande “burbujea” hacia el final de la lista.

Por ejemplo, al ordenar [5, 2, 4, 1, 3] se compararían 5 con 2 (se intercambian), luego 5 con 4 (se intercambian), etc. Es estable (no cambia el orden relativo de elementos iguales) y muy ineficiente para listas grandes: su complejidad en el peor y caso promedio es $O(n^2)$. Sólo resulta práctico para conjuntos pequeños o casi ordenados, o como ejemplo didáctico. Debe su simpleza a que utiliza solo intercambios sucesivos, pero su uso real es escaso fuera de contextos educativos.

Ventajas

Es un algoritmo de fácil comprensión e implementación, ideal para introducir conceptos básicos de algoritmos y programación. Además, es estable, lo que significa que conserva el orden relativo de los elementos con claves iguales. También permite una optimización sencilla: si durante una pasada no se realizan intercambios, el algoritmo puede finalizar antes de recorrer toda la lista, lo que mejora el rendimiento en listas casi ordenadas.

Desventajas

Tiene una complejidad temporal de $O(n^2)$ en el peor y promedio de los casos, lo cual lo hace ineficiente para listas grandes. Realiza muchas comparaciones e intercambios, lo que implica un alto costo computacional. Aunque se puede optimizar para listas casi ordenadas, sigue siendo considerablemente más lento que algoritmos como Quick Sort o Merge Sort en la práctica.

Casos de uso

Es adecuado principalmente en contextos educativos para enseñar los fundamentos del ordenamiento, debido a su simplicidad. También puede utilizarse en listas muy pequeñas o casi ordenadas, y cuando se requiere un algoritmo de ordenamiento estable y fácil de implementar, aunque su uso práctico es limitado en aplicaciones reales.

Ordenamiento por mezcla (Merge Sort)

Es un algoritmo de divide y vencerás que divide recursivamente la lista en dos mitades, ordena cada mitad y luego las fusiona. Conceptualmente: si la lista tiene ≤ 1 elemento ya está ordenada; en caso contrario se parten los datos en dos sublistas de tamaño aproximado medio, se ordena cada sublista recursivamente y luego se intercalan (mezclan) las dos sublistas ordenadas.

Por ejemplo, ordenar [8,3,7,4] implicaría dividir en [8,3] y [7,4], ordenar cada sublista ([3,8] y [4,7]) y luego fusionarlas dando [3,4,7,8]. Merge sort es estable y siempre ejecuta en tiempo $O(n \log n)$ (incluso en el peor caso). Es muy eficiente para conjuntos grandes y se usa cuando se necesita garantizar un orden rápido. Además, debido a que no trabaja in-place

(requiere espacio adicional para mezclar), es útil en ordenamiento externo (por ejemplo, al ordenar archivos grandes en disco). Fue ideado por John von Neumann en 1945.

Ventajas

Alto rendimiento garantizado, estable y naturalmente “paralelizable”. Es eficiente para grandes conjuntos de datos y útil para ordenamiento externo o en memoria secundaria. Funciona bien con estructuras de acceso secuencial (e.g. listas enlazadas)

Desventajas

Usa espacio extra proporcional al tamaño de la lista (no in-place). Tiene mayor sobrecarga de recursión y copias que Quick Sort, por lo que en la práctica suele ser más lento que Quick en tamaños moderados.

Casos de uso

Cuando se requiere un ordenamiento estable y rendimiento garantizado. Ideal para listas grandes, enlaces a datos, aplicaciones de ordenamiento externo y cuando la memoria adicional no es problema.

Ordenamiento rápido (Quicksort)

Es otro algoritmo divide y vencerás más complejo. Se elige un elemento pivote (por ejemplo, el primero o uno aleatorio) y se reorganizan los demás datos para que los menores que el pivote queden a su izquierda y los mayores a su derecha. Esto “particiona” la lista en dos sublistas (elementos $<$ pivote y $>$ pivote). Luego se aplica recursivamente quicksort a cada sublista.

Por ejemplo, al ordenar [5,2,9,1,6] con pivote 5, se agruparían [2,1] a la izquierda (menores) y [9,6] a la derecha (mayores), quedando 5 en posición. Luego se ordenan [2,1] → [1,2] y [9,6] → [6,9]. Quicksort no es estable por defecto y su rendimiento depende de la elección del pivote. En el caso promedio (pivote equitativamente dividido) tiene complejidad $O(n \cdot \log n)$, pero en el peor caso (por ejemplo, lista ya ordenada con pivote fijo en extremo) su complejidad empeora a $O(n^2)$.

No obstante, es uno de los métodos de ordenación más usados en la práctica por su bajo coste promedio y eficiencia en memoria (trabaja in-place sin espacio extra significativo). Se emplea a menudo para ordenamientos generales cuando no se garantizan situaciones patológicas. Dado que el pivote ideal divide la lista equilibradamente, las optimizaciones suelen centrarse en elegir pivotes aleatorios o medianos para evitar el peor caso.

Insertion Sort (ordenamiento por inserción)

Es parecido a ordenar un mazo de cartas: va tomando cada elemento y “lo inserta” en la posición correcta dentro de los anteriores ya ordenados. Trabaja bien con listas pequeñas o casi ordenadas.

Complejidad temporal: Mejor caso $O(n)$ (cuando la lista ya está casi ordenada); caso promedio y peor $O(n^2)$.

Complejidad espacial: $O(1)$ (in-place).

Ventajas

De implementación simple, estable e in-place. Es *adaptativo*: si los datos están casi ordenados, su rendimiento es casi lineal ($O(n)$ en el mejor caso). Requiere pocos intercambios si la lista está parcialmente ordenada.

Desventajas

Muy lento en listas grandes desordenadas (número de comparaciones e intercambios cuadrático). No es adecuado para grandes volúmenes de datos.

Casos de uso

Bueno para listas pequeñas o casi ordenadas. Se usa en subrutinas de otros algoritmos o cuando se recibe datos incrementalmente (online). También útil para mejorar ligeramente Timsort en sublistas pequeñas.

Algoritmos de búsqueda

Los algoritmos de búsqueda permiten encontrar datos dentro de una estructura. A continuación se examinan la búsqueda secuencial (lineal), la binaria y varios métodos de búsqueda en grafos.

Búsqueda lineal (secuencial)

Revisa los elementos uno a uno hasta hallar el objetivo o agotar la lista. Por ejemplo, para buscar “23” en [5, 18, 23, 7], se compararía cada elemento hasta encontrar el 23 en la tercera posición. No requiere que los datos estén ordenados. Es muy simple pero ineficiente para listas largas: en el peor caso efectúa n comparaciones ($O(n)$ de complejidad). Su caso promedio suele implicar $\sim n/2$ comparaciones, pero esto empeora a medida que crece la lista. Se utiliza únicamente cuando el conjunto de datos es pequeño o no se dispone de índices/estructuras avanzadas; de lo contrario, existen métodos mucho más rápidos. En resumen, la búsqueda lineal es trivial pero poco práctica a gran escala.

Ventajas

Es un algoritmo extremadamente simple de entender e implementar, no requiere que los datos estén ordenados previamente, y funciona con cualquier tipo de estructura lineal, como arreglos o listas.

Desventajas

Tiene una complejidad temporal de $O(n)$, lo que significa que su rendimiento disminuye significativamente con listas grandes. Es ineficiente en comparación con otros métodos cuando se trata de conjuntos de datos extensos.

Casos de uso

Es útil cuando se trabaja con listas pequeñas, datos no ordenados o estructuras en las que no es práctico aplicar otros algoritmos de búsqueda más complejos. También se emplea como base para validar o comparar otros métodos de búsqueda.

Figura 1

Ejemplo de búsqueda lineal

A screenshot of a code editor with a dark background and light-colored text. The code is a Python script for a linear search algorithm. It includes imports for 'time' and 'random', a function 'busqueda_lineal' that iterates through a list to find a target value, and a main execution block that generates a list of 100,000 random numbers, prompts the user for a search value, and prints the search results along with timing information.

```
1 import time
2 import random
3
4 def busqueda_lineal(lista, objetivo):
5     for i, elemento in enumerate(lista):
6         if elemento == objetivo:
7             return i
8     return -1
9
10 lista = [random.randint(1, 100) for _ in range(100000)]
11
12 buscar = int(input("¿Qué elemento deseas buscar? "))
13
14 tiempo_inicial = time.time()
15 posicion = busqueda_lineal(lista, buscar)
16 tiempo_final = time.time()
17
18 if posicion != -1:
19     print(f"Elemento '{buscar}' encontrado en la posición {posicion}.")
20 else:
21     print(f"Elemento '{buscar}' no encontrado en la lista.")
22
23 print(f"Tiempo inicial: {tiempo_inicial}")
24 print(f"Tiempo final: {tiempo_final}")
25 tiempo = tiempo_final - tiempo_inicial
26 print(f"Duración total: {tiempo}")
```

Fuente. Elaboración propia.

Búsqueda binaria

Es un algoritmo eficiente para listas ordenadas. Requiere que la colección esté previamente ordenada. Funciona dividiendo iterativamente el rango de búsqueda por la mitad: se compara el elemento central con el objetivo y, según el resultado, se descarta la

mitad de la lista donde no puede estar. Este proceso se repite en la mitad restante hasta encontrar el elemento o concluir que no existe.

Por ejemplo, para encontrar “50” en [10,20,30,40,50,60,70], se compara con el 40 (medio); como $50 > 40$, se busca en la sublista derecha [50,60,70], luego 50 con 60, etc. La búsqueda binaria requiere $O(\log n)$ comparaciones en el peor caso, mucho menor que $O(n)$. Es muy eficiente en grandes bases de datos ordenadas (como índices de bases de datos o directorios) donde las operaciones de comparación son rápidas.

Sin embargo, sólo se aplica si la lista ya está ordenada; de lo contrario habría que ordenarla antes, lo que puede no ser práctico si se busca solo una vez. La búsqueda binaria destaca por su rendimiento logarítmico y espacio constante ($O(1)$ adicional).

Ventajas

Extremadamente eficiente para listas ordenadas, con una complejidad logarítmica $O(\log n)$. Requiere un espacio constante en su versión iterativa, lo que la hace ideal para grandes conjuntos de datos. Es ampliamente utilizada en sistemas donde los datos ya están rígidamente estructurados.

Desventajas

Requiere que la lista esté previamente ordenada, lo que puede añadir un costo inicial de $O(n \log n)$ si no lo está. No es adecuada para estructuras dinámicas donde los datos cambian frecuentemente, ya que mantener el orden puede ser costoso.

Casos de Uso

Ideal para buscar en grandes bases de datos ordenadas, como índices de bases de datos, diccionarios o directorios. También se usa en algoritmos que requieren dividir el espacio de búsqueda, como en la resolución de problemas numéricos (por ejemplo, encontrar raíces de funciones).

Figura 2

Ejemplo de búsqueda binaria

A screenshot of a code editor window with a dark background and light-colored text. The code is a Python script that implements a binary search algorithm. It starts with importing 'time' and 'random'. A function 'busqueda_binaria' is defined, which takes a list and a target value. It uses a while loop to repeatedly divide the search range in half until the target is found or the range is empty. The script then generates a list of 1,000,000 random numbers, sorts it, and prompts the user to enter a number to search for. It measures the time taken for the search and prints the result and timing information.

```
1 import time
2 import random
3
4 def busqueda_binaria(lista, objetivo):
5     izquierda = 0
6     derecha = len(lista) - 1
7     while izquierda <= derecha:
8         medio = (izquierda + derecha) // 2
9         if lista[medio] == objetivo:
10             return medio
11         elif lista[medio] < objetivo:
12             izquierda = medio + 1
13         else:
14             derecha = medio - 1
15     return -1
16
17 # Genera una lista de un millón de números aleatorios entre 1 y 100 y la ordena
18 lista = [random.randint(1, 100) for _ in range(1000000)]
19 lista.sort()
20
21 buscar = int(input("¿Qué elemento deseas buscar? "))
22
23 tiempo_inicial = time.time()
24 posicion = busqueda_binaria(lista, buscar)
25 tiempo_final = time.time()
26
27 if posicion != -1:
28     print(f"Elemento '{buscar}' encontrado en la posición {posicion}.")
29 else:
30     print(f"Elemento '{buscar}' no encontrado en la lista.")
31
32 print(f"Tiempo inicial: {tiempo_inicial}")
33 print(f"Tiempo final: {tiempo_final}")
34 tiempo = tiempo_final - tiempo_inicial
35 print(f"Duración total: {tiempo}")
```

Fuente. Elaboración propia

Búsqueda en grafos

Cuando los datos están organizados en estructuras de red (grafos), se emplean algoritmos de recorrido o búsqueda de caminos. Un grafo consta de nodos (vértices) y aristas

(conexiones) con posibles costes asociados. El objetivo suele ser encontrar un camino entre dos nodos (por ejemplo, el más corto o de menor coste). Los más conocidos son:

Búsqueda en anchura (BFS)

Explora el grafo por niveles. Partiendo de un nodo inicial, primero visita todos sus vecinos inmediatos, luego los vecinos de éstos, y así sucesivamente. Se suele implementar con una cola (FIFO). BFS es un algoritmo no informado y garantiza encontrar el camino más corto en número de aristas en grafos no ponderados.

Por ejemplo, en una red social puede usarse para hallar el menor número de conexiones (amigos) que unen a dos personas. Su complejidad es $O(|V|+|E|)$, donde V es el conjunto de vértices y E el de aristas. Se emplea en problemas de distancia mínima en grafos no ponderados, en navegación de laberintos donde cada paso tiene igual costo, o para analizar redes (cobertura por niveles).

Ventajas

Garantiza encontrar el camino más corto en grafos no ponderados. Es completo (encuentra una solución si existe) y adecuado para explorar redes de manera sistemática. Funciona bien en grafos con baja profundidad.

Desventajas

Requiere una cantidad significativa de memoria para almacenar la cola y los nodos visitados, especialmente en grafos densos. No es eficiente en grafos con alta profundidad o cuando se buscan soluciones específicas en espacios grandes.

Casos de Uso

Útil en problemas como encontrar la distancia mínima en grafos no ponderados, análisis de redes sociales (por ejemplo, niveles de separación), resolución de laberintos, o en algoritmos de planificación y navegación

Figura 3

Ejemplo de búsqueda en anchura



```
1 from collections import deque
2
3 def busqueda_anchura(grafo, inicio, objetivo):
4     visitados = set()
5     cola = deque([inicio])
6     while cola:
7         nodo = cola.popleft()
8         if nodo == objetivo:
9             return True
10        if nodo not in visitados:
11            visitados.add(nodo)
12            cola.extend(vecino for vecino in grafo.get(nodo, []) if vecino not in visitados)
13    return False
```

Fuente: Elaboración propia

Búsqueda en profundidad (DFS)

Recorre el grafo siguiendo caminos “hacia abajo” hasta el final antes de retroceder. Desde el nodo inicial, se elige un vecino, luego un vecino de éste, y así sucesivamente hasta llegar a un nodo sin vecinos no visitados, momento en que retrocede (backtracking) para explorar ramas alternativas. DFS usa una pila o recursividad. No garantiza caminos más cortos, pero es útil para explorar completamente grafos.

Se utiliza en tareas como detectar ciclos, encontrar componentes conexas o generar recorridos completos. En términos formales, DFS es un algoritmo no informado que visita todos los nodos accesibles mediante recorridos profundos. Su complejidad es también $O(|V|+|E|)$, aunque en el peor caso podría recorrer árboles enormes (la complejidad temporal puede crecer exponencialmente con la profundidad en espacios de estados muy grandes).

Ventajas

Utiliza menos memoria que BFS en grafos con alta profundidad, ya que solo mantiene un camino en la pila. Es útil para explorar todas las ramas de un grafo y detectar estructuras como ciclos o componentes conexas. Es más rápido en grafos profundos o estrechos.

Desventajas

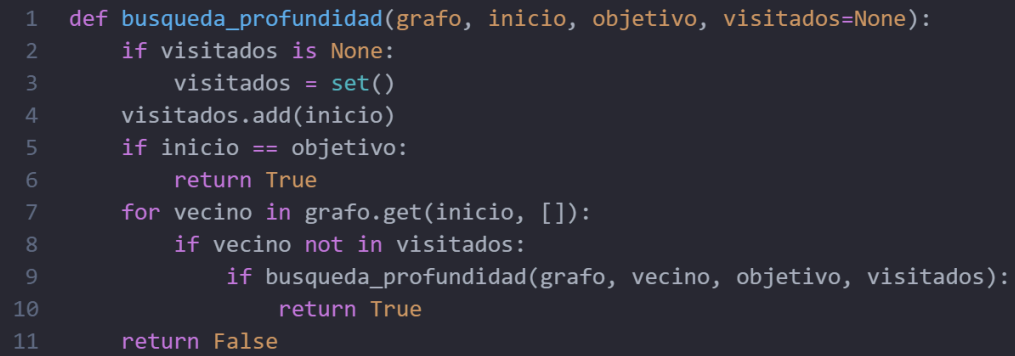
No garantiza encontrar el camino más corto. Puede quedar atrapado en ramas profundas en grafos muy extensos, lo que lo hace menos eficiente en ciertos casos. La implementación recursiva puede causar desbordamiento de pila en grafos extremadamente profundos.

Casos de Uso

Ideal para detectar ciclos, encontrar componentes conexas, generar recorridos completos, o resolver problemas como laberintos y búsqueda de caminos en grafos profundos.f

Figura 5

Ejemplo de búsqueda en profundidad



```
1 def busqueda_profundidad(grafo, inicio, objetivo, visitados=None):
2     if visitados is None:
3         visitados = set()
4     visitados.add(inicio)
5     if inicio == objetivo:
6         return True
7     for vecino in grafo.get(inicio, []):
8         if vecino not in visitados:
9             if busqueda_profundidad(grafo, vecino, objetivo, visitados):
10                 return True
11     return False
```

Fuente: Elaboración propia

Análisis a posterior

Algoritmos de ordenamiento

Tabla 1

Tabla de comparación de eficiencia medida en segundos (Algoritmos de ordenamiento)

Algoritmo	Mejor Caso	Peor Caso	Promedio
Bubble Sort	11.5 segundos	12.7 segundos	12.1 segundos
Merge Sort	0.140 segundos	0.160 segundos	0.153 segundos
Quick Sort	0.110 segundos	0.135 segundos	0.123 segundos

Fuente. Elaboración propia

Resultados

Para realizar el análisis, se realizaron pruebas para evaluar el rendimiento de bubble sort, merge sort y quick sort al ordenar listas de 5,000 elementos generadas aleatoriamente con números enteros entre 0 y 1000. Cada algoritmo fue sometido a 15 pruebas, midiendo el tiempo de ejecución para determinar el mejor, peor y promedio de los tiempos obtenidos. Es importante resaltar que los resultados varían según el hardware donde se haga la prueba.

Los resultados revelan que bubble sort, registró un tiempo promedio de 12 segundos, evidenciando la ineficiencia del algoritmo para listas de este tamaño. En contraste, merge sort y quick sort completaron las pruebas en 0.153 y 0.123 segundos respectivamente, lo cual llega a ser 80 veces más rápido. Quick sort mostró una ventaja en velocidad, mientras que merge sort se destacó por su estabilidad y consistencia mayor.

Algoritmos de búsqueda

Tabla 2

Tabla de comparación de eficiencia medida en segundos (Algoritmos de búsqueda)

Algoritmo	Mejor Caso (Primer elemento)	Peor Caso (Último elemento)	Promedio
Búsqueda Lineal	0.000010 segundos	0.003028 segundos	0.001663 segundos
Búsqueda binaria	0.000001 segundos	0.000005 segundos	0.000003 segundos
Búsqueda en anchura (BFS)	0.000031 segundos	0.082594 segundos	0.089821 segundos
Búsqueda en profundidad (DFS)	0.000005 segundos	0.000388 segundos	0.013627 segundos

Fuente. Elaboración propia

Resultados

En estas pruebas, se evaluó el rendimiento de los algoritmos de búsqueda más populares, en estructuras de datos con 50,000 elementos. Se utilizó listas para la búsqueda lineal y binaria, y grafos no dirigidos con 125,000 aristas para las dos últimas. Las pruebas midieron el tiempo de ejecución del mejor caso (primer elemento o nodo inicial), peor caso y promedio de tiempo (en 15 pruebas).

Se puede observar que la búsqueda lineal es significativamente más lenta debido a su forma de operar, ya que en grandes listas tiene que recorrer cada elemento uno por uno, favoreciendo más la búsqueda binaria para este tipo de listas. En grafos, la búsqueda en profundidad supera a la búsqueda en anchura, esto es debido a que DFS explora caminos profundos más rápidamente. Además, se observa que la búsqueda en anchura es ineficiente en grafos extensos, ya que requiere explorar más nodos y aristas.

Anexos

Repositorio: https://github.com/gague-uam/Investigacion_G1.git

Presentacion de canva:

https://www.canva.com/design/DAGrAxXaLNs/tsZt8r8RO4z-uyZq8qMetA/edit?utm_content=DAGrAxXaLNs&utm

Referencias

Kartik. (2025). Merge Sort - Data Structure and Algorithms Tutorials.

<https://www.geeksforgeeks.org/dsa/merge-sort/>

Navarro, A. (2016). Ordenamiento de burbuja – Algoritmos de ordenamiento.

[https://juncotic.com/ordenamiento-de-burbuja-algoritmos-de-ordenamiento/;](https://juncotic.com/ordenamiento-de-burbuja-algoritmos-de-ordenamiento/)

Patel, H. (2024). Algoritmo de ordenación rápida: una descripción general.

<https://builtin.com/articles/quicksort>

Mckee, A. (2024). Búsqueda lineal en Python: Guía para principiantes con ejemplos.

<https://www.datacamp.com/es/tutorial/binary-search-python>

Mckee, A. (2024). Búsqueda binaria en Python: guía completa para una búsqueda eficiente.

<https://www.datacamp.com/es/tutorial/binary-search-python>

Krishna, V. (2015). Graph Theory - Breadth First Search.

<https://www.hackerearth.com/practice/notes/graph-theory-breadth-first-search/>